

LAB 01 • CLASS 1 • SYSTEM & MEMORY

Memory → Living Site

Ghost learns who you are, saves it on purpose, and turns that memory into a real website you can send to anyone — then keeps the site in sync when you teach it something new.

DIFFICULTY EASIEST

TIME ~60 MIN

REINFORCES CLASS 1 • SYSTEM & MEMORY

YOU LEAVE WITH A LIVE DEPLOYED PERSONAL SITE

LEARNING OBJECTIVES

- Experience that Ghost's memory is **directed, not automatic** — you decide what is worth remembering, and Ghost can show you exactly what it saved and why.
- Turn saved memory into a **real, deployed artifact** — a personal site with a live URL, not a block of chat text.
- Prove the difference between a chat tab and an agent: Ghost **remembers across sessions** and **produces things that persist**.
- Practice the **four-beat method** — Prime, Prompt, Break & steer, Crystallize — on the gentlest possible task.
- Save your first reusable **workflow** so a single new fact can refresh the live site on demand.

Why this lab exists

Most people meet AI as a chat box: you ask, it answers, you close the tab, and everything it "knew" about you evaporates. Ghost is a different kind of tool. It has **persistent memory** across sessions and its own cloud workspace where it can build and **deploy real things**. This lab is the smallest possible demonstration of both facts working *together*.

You will have Ghost interview you, decide with you what is worth saving, and then generate a personal website *from that memory* — deployed to a live URL you can share. Then you will teach it one new fact and watch both the memory and the site update. By the end you will have felt the core mental model of Class 1 in a physical way: **memory is something you direct, and a Ghost that remembers can produce, not just answer**. Everything else in the course builds on this.

Prerequisites

- You completed **Lab 0** and have your bearings in the course.
- **Ghost is live in your Discord channel** and responding to you. You interact with Ghost entirely in Discord.
- A name you're comfortable putting on a public web page (a first name or handle is fine).

- Roughly five facts about yourself ready in your head: what you do, what you're into, one thing you're building or curious about. Don't over-prepare — the interview will pull these out of you.

The build — step by step

1

Prime — let Ghost interview you and save what matters

You are not going to dump a bio. You're going to let Ghost *ask*, and then have it commit specific facts to memory on purpose. This is the whole point of Class 1: memory is directed. Send this in Discord.

```
# PRIME: INTERVIEW + DIRECTED MEMORY
```

```
Interview me so you can remember me well. Ask me 5-6 short questions, one at a time, about who I am, what I do, what I'm into, and what I'm building right now. After we're done, save the durable facts to your memory about me — and then show me exactly what you saved and why you chose those facts over the throwaway details.
```

Answer the questions naturally. When Ghost reports back, read the "what I saved and why" list carefully — this is Ghost showing you its memory is a deliberate choice, not a transcript. If it saved something wrong or missed something that matters to you, say so now: *"Drop the part about X, and also remember that I..."*

2

Prompt — turn memory into a deployed site

Now the payoff. Ask Ghost to build the site **from memory** — deliberately giving it no facts in this message. If the site comes out right, it's because Ghost genuinely remembered.

```
# PROMPT: BUILD + DEPLOY FROM MEMORY ALONE
```

```
Using ONLY what you remember about me — don't ask me for anything new — build and deploy a personal website for me. Clean, single page, my name up top, a short intro, and sections that reflect the facts you saved. Deploy it and give me the live URL.
```

Ghost will work in its own cloud workspace and hand you a real link. Open it. This is your first proof that Ghost **remembers and produces** — the page in front of you was assembled from facts you deliberately saved a few minutes ago, in a session you could have closed and reopened.

Break & steer — change one fact and expect the site to follow

Here's the forced failure that teaches the real lesson. Teach Ghost something that *contradicts* what it already knows — a new role, a new city, a finished project — and demand that both memory and the live site reflect it. **Do not restart. Do not rebuild from scratch.** Steer.

```
# BREAK & STEER: CONTRADICT A FACT, EXPECT SYNC
```

```
Update what you know about me: I just started a new role as a product
designer (I was an engineer before). Change your memory to match, then
update the SAME live site so it reflects this. Give me the URL again.
```

Two outcomes are possible, and both are lessons:

- **The site updates cleanly** at the same URL — memory and artifact stayed in sync. Perfect.
- **The site still shows the old fact** even though Ghost says memory changed. That means Ghost *hardcoded* your facts into the page instead of generating the page *from* memory. Debug it, don't restart:

```
# STEER THE FIX, DON'T REBUILD
```

```
The live page still says "engineer." Don't start over. The problem is the
page has my details baked in as static text. Regenerate the page so its
content is driven by your saved memory of me, then redeploy to the same URL
and confirm it now reads "product designer."
```

Crystallize — freeze the update loop into a workflow

You just did a valuable thing twice (change a fact → refresh the site). Turn it into a reusable **workflow** saved to your Ghost, so you never have to spell out the steps again.

```
# CRYSTALLIZE: SAVE A REUSABLE "UPDATE-SITE" WORKFLOW
```

```
Save a workflow called "update-site." When I tell you a new fact about myself: (1) update your saved memory of me, (2) regenerate my personal site from that memory — never hardcode facts into the page, (3) add or refresh a small "Last updated" note with today's date, and (4) redeploy to the same URL and send me the link. Confirm it's saved.
```

From now on, "remember I started learning Japanese — run update-site" is a complete instruction. That compression is what makes an agent feel like it's *yours*.

The forced failure: static text vs. living memory

The failure this lab is designed to provoke is subtle and worth naming clearly. When you ask for a website, the fastest thing for any model to do is paste your facts directly into the page's text. It looks identical to the "right" answer — until a fact changes. A page with baked-in text is a snapshot; a page generated from memory is *alive*. Class 1 is about noticing that difference and insisting on the living version.

BREAK & STEER — FAILURE MODES TO EXPECT

- **Hardcoded facts.** Memory updates but the page doesn't. Fix by steering Ghost to drive page content *from* memory and redeploy to the same URL (Step 3). This is the headline lesson — don't skip past it if it happens.
- **New URL on every change.** Ghost redeploys to a fresh link each time. Steer: "redploy to the same URL so my link never changes."
- **Silent memory.** Ghost rebuilds the site but never confirms what it changed in memory. Ask it to echo the updated fact back so you can trust the two are in sync.
- **Over-collection.** Ghost saves the entire interview verbatim. Push back — directed memory means the *useful* facts, not the transcript.

Deliverable

WHAT YOU MUST BE ABLE TO SHOW

- ☐ A **live URL** to a personal site Ghost built and deployed for you.
- ☐ The site's content visibly reflects **facts Ghost saved to memory** — not facts you pasted into the build request.
- ☐ A **before/after demo**: one new fact caused the live site to change, at the same URL.
- ☐ A saved **"update-site" workflow** on your Ghost that reruns the change → refresh loop.
- ☐ You can explain, in one sentence, why a chat tab could not have produced this.

Stretch goals

1. **Portfolio from memory.** Tell Ghost about two or three projects and have it add a projects/portfolio section — pulled from memory, so it grows as you tell it about new work.
2. **Restyle on request.** Ask Ghost to restyle the site ("make it dark and minimal," "warmer, more editorial") without changing any facts — proving presentation and memory are separate concerns.
3. **Change log.** Have the "update-site" workflow keep a running "Last updated" note or a tiny list of recent changes, so the living site wears its own history.
4. **Web-sourced touch.** Give Ghost a public link about your work and ask it to fold one accurate detail into the site — a first taste of Ghost's web access feeding the artifact.

FACILITATOR NOTES

- This is the confidence-builder. Keep energy high and let students feel the "it remembered me" moment in Step 2 before you complicate anything.
- The forced failure in Step 3 is the pedagogical core. If a student's site updates cleanly on the first try, have them deliberately probe: ask Ghost whether the page reads from memory or has facts baked in, and make the distinction explicit.
- Watch for students who paste their full bio into the Step 2 build request — that defeats the "from memory alone" proof. Redirect them to trust the interview.
- Reinforce vocabulary: *directed memory*, *artifact*, *workflow*, *same-URL redeploy*. These terms recur in every later lab.
- Don't promise scheduling or self-wake behavior — updates in this lab are user-triggered. Keep claims inside Ghost's real envelope: memory, cloud workspace, deploy, web access, saved workflows.